



RAG ~ 數位員工

我的開發環境是AMD Ryzen AI Max+ 395 (128GB)的
Windows11上有LM Studio 可以運行Qwen3.6-35B-A3B +
LINUX(VM) 的 Hermes Agent 運行的環境



這套硬體與架構配置非常強大！AMD Ryzen AI Max+ 395 擁有 128GB 的統一記憶體 (Unified Memory)，能輕鬆順暢地執行像 Qwen3.6-35B-A3B 這類高效能且支援 Agentic 工作流程的 MoE 模型。

為了實現「只在特定知識庫內檢索與回答，不幻覺、可隨時擴充」的專屬數位員工，最穩健且符合 Hermes Agent 工具呼叫 (Tool Calling) 機制的架構，是在 Linux VM 內建立一個 RAG 微服務（基於 LightRAG / Qdrant + FastMCP），並將其作為 Tool/Skill 掛載給 Hermes Agent。

以下是針對小白設計、步驟明確的實作流程：

系統架構總覽

[Windows 11 主機]

- └─ LM Studio (提供 LLM 服務，API 埠號 1234)
- └─ 運行 Qwen3.6-35B-A3B

[Linux VM (WSL2 或 Hyper-V/VirtualBox)]

- └─ 知識庫服務 (Qdrant 向量資料庫 + RAG 檢索 API)
- └─ Hermes Agent
 - └─ 透過 MCP (Model Context Protocol) 呼叫 RAG 工具進行資料檢索

完整實作步驟

第一步：Windows 端 LM Studio 設定

1. 在 Windows 啟動 LM Studio 並載入 Qwen3.6-35B-A3B 模型。
2. 切換至左側 Developer / Local Server 分頁。

3. 將預設埠號設為 1234 ，開啟 **Cross-Origin Resource Sharing (CORS)** 。
4. 點擊 **Start Server** 。
5. **關鍵步驟（取得主機 IP）：**

在 Windows 開啟 CMD / PowerShell ，輸入 `ipconfig` 。

找到 IPv4 位址 （例如 192.168.1.100 或 WSL 虛擬網卡 IP 172.x.x.x ）。

測試測試 API：在 Linux VM 執行

`curl http://<Windows_IP>:1234/v1/models` ，若能返回 JSON 即代表連通。

第二步：Linux VM 環境準備與知識庫建置

建議使用 Python 輕量且檢索極強的 **Qdrant** 作為向量資料庫。

1. 安裝必要套件

在 Linux VM 終端機執行：

Bash



```
# 更新並安裝 Python 環境
sudo apt update && sudo apt install -y python3-pip python3-venv docker.io

# 啟動 Qdrant 向量資料庫（Docker 方式最簡單）
sudo docker run -d -p 6333:6333 -p 6334:6334 \
  -v $(pwd)/qdrant_storage:/qdrant/storage:z \
  qdrant/qdrant
```

2. 建置知識庫匯入與檢索腳本 (rag_service.py)

建立一個資料夾 `mkdir ~/knowledge_agent && cd ~/knowledge_agent` ，創建 `rag_service.py`：

Python



```
import os
from sentence_transformers import SentenceTransformer
from qdrant_client import QdrantClient
from qdrant_client.models import Distance, VectorParams, PointStruct

# 1. 初始化嵌入模型 (使用繁中/跨語言效果優異的 Embedding 模型)
encoder = SentenceTransformer('BAAI/bge-m3')

# 2. 連接本地 Qdrant
client = QdrantClient(host="localhost", port=6333)
COLLECTION_NAME = "domain_knowledge"

# 初始化 Collection
if not client.collection_exists(COLLECTION_NAME):
    client.create_collection(
        collection_name=COLLECTION_NAME,
        vectors_config=VectorParams(size=1024, distance=Distance.COSINE),
    )

def ingest_document(file_path):
    """將文本檔案切割並寫入向量資料庫"""
    with open(file_path, 'r', encoding='utf-8') as f:
        text = f.read()

    # 簡單按段落切分 (小白實作版，後續可擴充為依 Token 切分)
    chunks = [c.strip() for c in text.split('\n\n') if len(c.strip()) > 10]

    points = []
    for idx, chunk in enumerate(chunks):
        vector = encoder.encode(chunk).tolist()
        points.append(PointStruct(
            id=idx,
            vector=vector,
            payload={"content": chunk, "source": os.path.basename(file_path)}
        ))

    client.upsert(collection_name=COLLECTION_NAME, points=points)
    print(f"成功匯入 {len(chunks)} 筆文件區塊!")

def search_knowledge(query, top_k=3):
    """檢索與問題最相關的文字"""
    query_vector = encoder.encode(query).tolist()
    results = client.search(
        collection_name=COLLECTION_NAME,
        query_vector=query_vector,
        limit=top_k
    )

    contexts = [hit.payload['content'] for hit in results if hit.score > 0.35]
```

```
if not contexts:
    return "【系統提示】：知識庫中無相關資料。"

return "\n---\n".join(contexts)

if __name__ == "__main__":
    # 測試：建立 docs 資料夾，放一個 test.txt
    os.makedirs("docs", exist_ok=True)
    with open("docs/test.txt", "w", encoding="utf-8") as f:
        f.write("本公司客服政策：退貨需在購買後 14 天內提出，且必須保留原包裝與發票。")

    ingest_document("docs/test.txt")
    print("測試查詢結果：")
    print(search_knowledge("退貨期限是多久？"))
```

在 VM 執行一次測試：

Bash



```
pip install sentence-transformers qdrant-client
python3 rag_service.py
```

第三步：將 RAG 打包為 FastMCP (Hermes Agent 的工具擴充)

Hermes Agent 原生支援 MCP (Model Context Protocol)，我們只需將 RAG 檢索邏輯寫成一個 MCP Server。

在 VM 創建 mcp_rag_server.py：

Python



```
from mcp.server.fastmcp import FastMCP
from rag_service import search_knowledge, ingest_document
import os

# 初始化 MCP Server
mcp = FastMCP("DomainKnowledgeBase")

@mcp.tool()
def query_knowledge_base(query: str) -> str:
```

```

"""
    用於查詢專屬領域知識庫（如公司產品客服、特定法律條文等）。
    當使用者詢問特定領域業務時，必須優先調用此工具。
"""

return search_knowledge(query, top_k=3)

@mcp.tool()
def add_new_document(file_path: str) -> str:
    """
    擴充知識庫：將指定路徑的 txt 檔案寫入知識庫。
    """
    if os.path.exists(file_path):
        ingest_document(file_path)
        return f"檔案 {file_path} 已成功擴充入知識庫！"
    return "檔案路徑不存在。"

if __name__ == "__main__":
    mcp.run(transport="stdio")

```

第四步：配置 Hermes Agent 並綁定邊界限制

1. 設定 Hermes Agent 連接 LM Studio

在 Linux VM 的 Hermes Agent 設定檔（通常為 `~/.hermes/config.yaml` 或環境變數）中調整模型端點：

YAML



```

llm:
  provider: "openai"
  base_url: "http://<Windows_IP>:1234/v1" # 填入 Windows 的 IP
  api_key: "lm-studio" # LM Studio 任意字串即可
  model: "qwen3.6-35b-a3b"

# 載入 MCP 工具
mcp_servers:
  knowledge_rag:
    command: "python3"
    args:
      - "/home/user/knowledge_agent/mcp_rag_server.py"

```

2. 定義數位員工 Prompt (嚴格鎖定知識庫)

在 Hermes Agent 的 System Prompt（或 Persona 系統設定檔）寫入以下原則，**確保系統「只」依靠知識庫回答**：

數位員工角色指令：

你是一名專屬領域的專業諮詢客服。

1. 當收到使用者提問時，你必須強制先呼叫 `query_knowledge_base` 工具。
2. 嚴格原則：你只能根據 `query_knowledge_base` 返回的內容回答問題。
3. 若檢索結果回傳「知識庫中無相關資料」，請直接回覆：「抱歉，目前專屬知識庫內並無相關規範/資料可供查詢，我無法為您解答。」切勿使用你原本的常識自行捏造或通俗回答。

第五步：日常營運與擴充知識庫 (小白維護指南)

當需要新增客服資料或法律條文時，只需要：

1. **直接擴充**：將新的 `.txt` 或 `.md` 檔案放到 VM 的 `~/knowledge_agent/docs/` 目錄。
2. **透過對話擴充**：在 Hermes Agent 的對話視窗直接對數位員工說：

"請幫我把 `/home/user/knowledge_agent/docs/2026新退款條例.txt` 擴充到知識庫中。"

Hermes Agent 便會自動調用 `add_new_document` 工具將新資料嵌入向量庫，完成無縫擴充！

需要為您補全更完整的文本自動切塊與 PDF/Word 解析模組嗎？